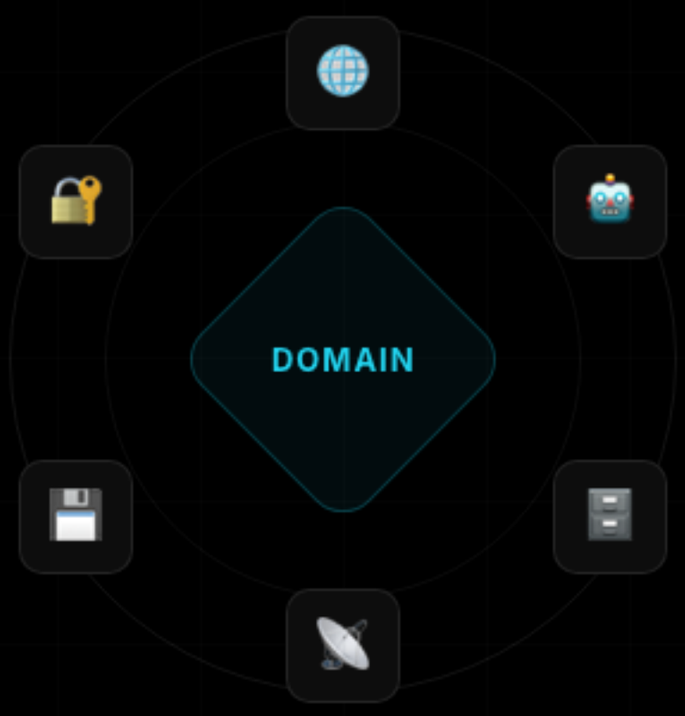
FREE CHEAT SHEET



Hexagonal Architecture Cheat Sheet

for GenAI Engineers

From the video: "Why Coupling Kills Your GenAI Projects"

- Architecture Patterns
- Ports & Adapters
- Coupling Costs
- GenAI Stack

Nil Monfort
AI Solutions Architect
youtube.com/@nilmonfort

The Architecture Spectrum + Ports & Adapters

3 ARCHITECTURE OPTIONS

1 Script Approach

One file, everything glued. **Fast to write, zero complexity.** Great for prototypes. Dead after a weekend.

Fast start | Zero reuse | Untestable

2 Layered Architecture

Horizontal layers: **presentation, business, persistence.** Better separation, but dependencies flow downward. Business logic still coupled to the database layer.

Better separation | Deps flow down | DB coupling

3 Hexagonal Architecture

Dependencies point **INWARD.** Domain at center knows nothing about the outside world. Everything adapts to the core, not the other way around.

Inward deps | Swappable | Testable

DEPENDENCY DIRECTION COMPARISON

Layered: UI → Business → DB

Hexagonal: API → Use Cases → Domain ← DB

PORT EXAMPLE (PYTHON)

```python
class LLMPort(ABC):
    # Contract: "I need something that
    # can generate text"
    @abstractmethod
    def generate(self, prompt: str) -> str:
        ...
```

ADAPTER EXAMPLE (PYTHON)

```python
class OpenAIAdapter(LLMPort):
    # Implementation: talks to OpenAI
    def generate(self, prompt: str) -> str:
        return self.client.chat(prompt)
```

PORTS & ADAPTERS VOCABULARY

Port
A contract. "I need something that can do X" without caring how. e.g. `LLMPort`, `ConversationRepositoryPort`

Adapter
The actual implementation that talks to a specific external system. e.g. `OpenAIAdapter` implements `LLMPort`

Domain
The actors of your application. Pure business entities with zero external dependencies. Knows nothing about APIs, databases, or LLMs.

Use Cases
The verbs, the actions they perform. Orchestrate the flow between domain and ports. **Don't implement, coordinate.**

Infrastructure
The executor, the tools that carry out actions. All adapters live here: API routes, DB clients, LLM SDKs, caches.

PORT TYPES

Driving Ports (Primary)
How the outside world triggers your app

REST API | CLI | Discord bot | WebSocket | gRPC

Driven Ports (Secondary)
What your app needs from outside

LLM provider | Database | Cache | Storage | Auth | Observability

LAYER MAPPING

LAYER	RESPONSIBILITY
Domain	Entities, value objects, business rules
Application	Use cases, port interfaces, DTOs
Infrastructure	Adapters, API routes, DB clients

The Golden Rule: Dependencies always point inward. Domain never imports from infrastructure. Infrastructure depends on domain, never the reverse.

The 4 Coupling Costs + GenAI Stack Reality

THE 4 COUPLING COSTS

COST #1
Can't Switch Providers
EU client needs data residency? **Find every OpenAI import, rewrite each call, pray.** One provider change touches dozens of files.

COST #2
Can't Test Logic
Need real MongoDB, real API key, **real money per test run.** Deploy Friday, pray Saturday. No mocks possible.

COST #3
Can't Reuse Logic
Same chatbot for REST, Discord, Slack, CLI? **Logic trapped in one class.** Copy-paste becomes the "architecture".

COST #4
Can't Optimize Cost
Stuck paying GPT-4 when GPT-3.5 works. **Switching means rewriting.** Your bill grows, your architecture fights back.

GENAI STACK REALITY

Minimum 7 External Dependencies

LLM Provider	OpenAI / Anthropic / Gemini
Embedding Model	text-embedding-3 / Cohere
Vector Database	Pinecone / Qdrant / pgvector
Storage	S3 / GCS / local
Cache	Redis / Memcached
Auth	Clerk / Auth0 / Cognito
Observability	LangSmith / Langfuse

> *"Your app has more API keys than features."*

Business Logic (YOUR CODE) vs Infrastructure & Integrations (EVERYTHING ELSE)

Your actual business logic is **tiny** compared to all this infrastructure. Without ports and adapters, every provider change cascades through your entire codebase.

COUPLED VS DECOUPLED

TIGHTLY COUPLED

```
class Chatbot:
  def __init__(self):
    self.client = OpenAI(api_key="sk-...")
    self.db = MongoClient("mongo://...")
    # Swap provider = rewrite class
    # Test = need real connections
```

HEXAGONAL (DECOUPLED)

```
class ChatUseCase:
  def __init__(self,
    llm: LLMPort,
    repo: ConversationPort):
    # Swap = inject different adapter
    # Test = inject fake
```

THE PAYOFF

3 hrs
Provider Swap (Decoupled)
One file, under 50 lines changed

Weeks
Provider Swap (Coupled)
Dozens of files, cascading rewrites

WHO USES THIS: Netflix (microservices) · AWS (Prescriptive Guidance) · Shopify (payment decoupling)

"In AI, being a week late is basically being retired. The architecture that lets you swap providers in hours instead of weeks is the one that survives."

WANT MORE LIKE THIS?

Part 2 covers the complete implementation

Every concept from this cheat sheet, built step-by-step in Python code.

Subscribe for Part 2
youtube.com/@nilmonfort

Source Code on GitHub
github.com/nilmonfort/ai-engineering-patterns

Watch the Full Video
Hexagonal Architecture Part 1: Why Coupling Kills Your GenAI Projects

Nil Monfort
The Production AI Engineer